

```

"""Tests for src.alerts: Alert and AlertRule."""

from datetime import datetime

import pytest

from src.alerts import Alert, AlertRule
from src.readings import SensorReading

# -----
# Helpers
# -----

def make_reading(sensor_id="temp-1", value=0.0, unit="C", timestamp=None):
    """Build a SensorReading with sensible defaults for tests."""
    if timestamp is None:
        timestamp = datetime(2026, 1, 1, 12, 0, 0)
    return SensorReading(
        sensor_id=sensor_id, value=value, unit=unit, timestamp=timestamp
    )

# -----
# Alert: severity validation
# -----

class TestAlertSeverityValidation:
    """Tests for Alert's severity validation in the constructor."""

    @pytest.mark.parametrize("severity", ["LOW", "MEDIUM", "HIGH", "CRITICAL"])
    def test_valid_severity_accepted(self, severity):
        """Each of the four allowed severities should construct cleanly."""
        alert = Alert(
            sensor_id="temp-1",
            severity=severity,
            message="test message",
            timestamp=datetime(2026, 1, 1),
        )
        assert alert.severity == severity

    @pytest.mark.parametrize(
        "bad_severity", ["low", "Medium", "URGENT", "", "HIGHEST", None]
    )

```

```

def test_invalid_severity_raises_value_error(self, bad_severity):
    """Invalid or wrongly-cased severities must raise ValueError."""
    with pytest.raises(ValueError):
        Alert(
            sensor_id="temp-1",
            severity=bad_severity,
            message="test message",
            timestamp=datetime(2026, 1, 1),
        )

# -----
# Alert: ordering
# -----

class TestAlertOrdering:
    """Tests for Alert.__lt__ and severity-based sorting."""

    def test_sorting_mixed_severities(self):
        """Sorting a list of mixed-severity alerts should order LOW..CRITICAL."""
        ts = datetime(2026, 1, 1)
        critical = Alert("s1", "CRITICAL", "m", ts)
        high = Alert("s2", "HIGH", "m", ts)
        medium = Alert("s3", "MEDIUM", "m", ts)
        low = Alert("s4", "LOW", "m", ts)

        shuffled = [high, low, critical, medium]
        ordered = sorted(shuffled)

        assert [a.severity for a in ordered] == [
            "LOW",
            "MEDIUM",
            "HIGH",
            "CRITICAL",
        ]

    def test_lt_direct_comparison(self):
        """Direct __lt__ comparisons should respect the severity rank."""
        ts = datetime(2026, 1, 1)
        low = Alert("s1", "LOW", "m", ts)
        critical = Alert("s2", "CRITICAL", "m", ts)

        assert low < critical
        assert not critical < low

    def test_max_picks_highest_severity(self):
        """max() over alerts should return the most severe one."""

```

```

        ts = datetime(2026, 1, 1)
        alerts = [
            Alert("s1", "LOW", "m", ts),
            Alert("s2", "CRITICAL", "m", ts),
            Alert("s3", "MEDIUM", "m", ts),
        ]
        assert max(alerts).severity == "CRITICAL"

# -----
# AlertRule.check(): in range
# -----

class TestAlertRuleInRange:
    """Tests for AlertRule.check() when the reading is within bounds."""

    def test_value_within_range_returns_none(self):
        """A value comfortably inside [min, max] should produce no alert."""
        rule = AlertRule(sensor_id="temp-1", min_threshold=0.0, max_threshold=100)
        reading = make_reading(sensor_id="temp-1", value=50.0)
        assert rule.check(reading) is None

    def test_value_exactly_on_boundary_returns_none(self):
        """Boundary values (inclusive range) should not trigger an alert."""
        rule = AlertRule(sensor_id="temp-1", min_threshold=0.0, max_threshold=100)
        at_min = make_reading(sensor_id="temp-1", value=0.0)
        at_max = make_reading(sensor_id="temp-1", value=100.0)
        assert rule.check(at_min) is None
        assert rule.check(at_max) is None

# -----
# AlertRule.check(): severity tiers
# -----

class TestAlertRuleSeverityTiers:
    """Tests for severity selection at MEDIUM/HIGH/CRITICAL boundaries.

    Rule range is [0, 100], so max_threshold = 100 is the breached
    threshold for all over-range cases below. Deviation fraction is
    |value - 100| / 100.
    """

    def setup_method(self):
        """Create a shared rule with thresholds [0, 100] for each test."""
        self.rule = AlertRule(
            sensor_id="temp-1", min_threshold=0.0, max_threshold=100.0

```

```

)

def test_medium_just_over_threshold(self):
    """A small overshoot (5%) should be MEDIUM."""
    reading = make_reading(sensor_id="temp-1", value=105.0)
    alert = self.rule.check(reading)
    assert alert is not None
    assert alert.severity == "MEDIUM"

def test_medium_at_exact_10_percent_boundary(self):
    """Exactly 10% over should still be MEDIUM (inclusive boundary)."""
    reading = make_reading(sensor_id="temp-1", value=110.0)
    alert = self.rule.check(reading)
    assert alert.severity == "MEDIUM"

def test_high_just_over_10_percent_boundary(self):
    """Just past 10% (e.g. 15%) should be HIGH."""
    reading = make_reading(sensor_id="temp-1", value=115.0)
    alert = self.rule.check(reading)
    assert alert.severity == "HIGH"

def test_high_at_exact_25_percent_boundary(self):
    """Exactly 25% over should still be HIGH (inclusive boundary)."""
    reading = make_reading(sensor_id="temp-1", value=125.0)
    alert = self.rule.check(reading)
    assert alert.severity == "HIGH"

def test_critical_just_over_25_percent_boundary(self):
    """Just past 25% (e.g. 30%) should be CRITICAL."""
    reading = make_reading(sensor_id="temp-1", value=130.0)
    alert = self.rule.check(reading)
    assert alert.severity == "CRITICAL"

def test_critical_far_under_minimum(self):
    """A value far below the minimum threshold should be CRITICAL."""
    reading = make_reading(sensor_id="temp-1", value=-50.0)
    alert = self.rule.check(reading)
    assert alert.severity == "CRITICAL"

def test_medium_just_under_minimum(self):
    """A small undershoot below a nonzero minimum should be MEDIUM."""
    rule = AlertRule(sensor_id="temp-1", min_threshold=100.0, max_threshold=2
    reading = make_reading(sensor_id="temp-1", value=95.0) # 5% under min
    alert = rule.check(reading)
    assert alert.severity == "MEDIUM"

```

```
# -----
# AlertRule.check(): non-matching sensor
# -----

class TestAlertRuleSensorMismatch:
    """Tests for AlertRule.check() ignoring readings from other sensors."""

    def test_non_matching_sensor_id_returns_none(self):
        """A reading from a different sensor must be ignored, even out of range."
        rule = AlertRule(sensor_id="temp-1", min_threshold=0.0, max_threshold=100
        reading = make_reading(sensor_id="temp-2", value=999.0)
        assert rule.check(reading) is None
```